Day 4

IN, BETWEEN, LIKE and NULL handling

Phase	Time	Dialect	Dataset
1: Foundations	60-90 minutes	PostgreSQL (SQLite notes included)	customers, products, orders, order_items

Learning objectives

- Match a column against a list of values with **IN** and **NOT IN**.
- Select a range of numbers or dates with **BETWEEN**, and know it includes both ends.
- Search for text patterns with **LIKE** using the **%** and _ wildcards.
- Handle missing data correctly with **IS NULL** and **IS NOT NULL**, and avoid the classic NULL traps.

Before you start

Day 3 gave you `WHERE` with comparison operators and `AND`/`OR`/`NOT`. Today's tools are shortcuts and refinements for the same `WHERE` clause. You need your `sqlcourse` database loaded (Day 1) and an open query window in pgAdmin (or DB Browser for SQLite).

```
-- Reminder of the shared schema
-- customers(customer_id PK, name, email, city, signup_date)
-- products(product_id PK, name, category, price)
-- orders(order_id PK, customer_id FK, order_date, status)
-- order_items(order_id FK, product_id FK, quantity, unit_price)
```

Your values may differ from mine. Where a lesson query uses a city, a category, or a name, replace it with a value you actually see in your tables (Guided Practice Step 0 shows you how). Result counts I state are only the ones fixed by the dataset design: ids, and the NULL email for customer 7.

1. Concept (15 min)

1.1 IN: match any value in a list

Suppose you want customers 1, 2 and 3. With what you know, you would write three ORs. `IN` says the same thing in one line.

```
-- Long way
SELECT * FROM customers
WHERE customer_id = 1 OR customer_id = 2 OR customer_id = 3;

-- Short way (identical result)
SELECT * FROM customers
WHERE customer_id IN (1, 2, 3);
```

- Text values go in single quotes: `city IN ('Delhi', 'Mumbai')`.
- `NOT IN` is the opposite: keep rows whose value is *not* in the list.

1.2 BETWEEN: a range, ends included

`BETWEEN low AND high` keeps values from low to high, **including both low and high**. It is shorthand for `>= low AND <= high`.

```
SELECT * FROM products
WHERE product_id BETWEEN 103 AND 106;   -- returns 103, 104, 105, 106
```

- Put the smaller value first. `BETWEEN 106 AND 103` returns zero rows, with no error.
- Works on numbers, dates and text. For dates: `signup_date BETWEEN '2024-01-01' AND '2024-06-30'`.
- `NOT BETWEEN` keeps everything outside the range.

> **Date warning.** If a column stores a time as well as a date (a timestamp), `BETWEEN '2024-06-01' AND '2024-06-30'` stops at midnight at the start of June 30 and misses most of that day. For timestamps use `>= '2024-06-01' AND < '2024-07-01'`. For plain DATE columns BETWEEN is fine.

1.3 LIKE: search by pattern

When you know part of a text value, use `LIKE` with two wildcards:

Wildcard	Meaning	Example	Matches
%	Any number of characters (including none)	name LIKE 'A%'	Names starting with A
%	(at both ends)	name LIKE '%an%'	Names containing 'an' anywhere
%	(at the start)	email LIKE '%.com'	Emails ending in .com
_	Exactly one character	name LIKE '_a%'	Names whose 2nd letter is a

> **Dialect difference.** In **PostgreSQL**, LIKE is **case-sensitive**: 'A%' will not match 'anil'. Use `ILIKE` for case-insensitive matching. In **SQLite**, LIKE ignores case for English letters and ILIKE does not exist. Everything else in this lesson works on both.

1.4 NULL: a missing value

NULL does not mean zero or an empty string. It means *unknown or missing*. In our data, customer 7 has no email, so their email is NULL. Because NULL is "unknown", any comparison with it is also unknown, and WHERE only keeps rows where the condition is true.

```
SELECT * FROM customers WHERE email = NULL;     -- 0 rows. Never works!
SELECT * FROM customers WHERE email IS NULL;    -- customer 7
SELECT * FROM customers WHERE email IS NOT NULL;-- everyone else
```

Three consequences to memorise:

- `= NULL` and `<> NULL` never match anything. Always use `IS NULL` / `IS NOT NULL`.
- `email LIKE '%gmail.com'` never matches customer 7, and neither does `email NOT LIKE '%gmail.com'`. A NULL fails both the pattern and its opposite.
- `NOT IN (...)` returns **no rows at all** if the list contains a NULL. You will see this in the stretch goal.

2. Guided Practice (20 min)

Type each query yourself rather than pasting. Predict the result first, then run it and check.

Step 0. Look at your data so you know which values exist.

```
SELECT * FROM customers;
SELECT * FROM products;
```

Step 1. IN with numbers.

```
SELECT name, city
FROM customers
WHERE customer_id IN (1, 2, 3);
```

Expect exactly 3 rows.

Step 2. NOT IN: the opposite.

```
SELECT name, city
FROM customers
WHERE customer_id NOT IN (1, 2, 3);
```

Expect 5 rows (customers 4 to 8).

Step 3. IN with text. Replace the cities with two that appear in your table.

```
SELECT name, city
FROM customers
WHERE city IN ('Delhi', 'Mumbai');
```

Step 4. BETWEEN on ids, then on price.

```
SELECT product_id, name
FROM products
WHERE product_id BETWEEN 103 AND 106;

SELECT name, price
FROM products
WHERE price BETWEEN 10 AND 50;
```

The first returns 4 rows (103, 104, 105, 106: both ends included). Adjust the price range to suit your data.

Step 5. NOT BETWEEN.

```
SELECT name, price
FROM products
WHERE price NOT BETWEEN 10 AND 50;
```

Step 6. BETWEEN on dates. Pick a window that fits the dates you saw in Step 0.

```
SELECT name, signup_date
FROM customers
WHERE signup_date BETWEEN '2024-01-01' AND '2024-12-31';
```

Step 7. LIKE patterns. Run all four and compare.

```
SELECT name FROM customers WHERE name LIKE 'A%';
SELECT name FROM customers WHERE name LIKE '%a';
SELECT name FROM customers WHERE name LIKE '%an%';
SELECT name FROM customers WHERE name LIKE '_a%';
```

In PostgreSQL, try swapping LIKE for ILIKE in the first query and see whether more rows appear.

Step 8. The NULL trap. Run both and compare.

```
SELECT customer_id, name FROM customers WHERE email = NULL;
SELECT customer_id, name FROM customers WHERE email IS NULL;
```

The first returns nothing and shows no error, which is why this bug is easy to miss. The second returns customer 7.

Step 9. IS NOT NULL.

```
SELECT customer_id, name, email
FROM customers
WHERE email IS NOT NULL;
```

Expect 7 rows.

Step 10. Combine everything. Note the parentheses.

```
SELECT customer_id, name, email
FROM customers
WHERE customer_id BETWEEN 1 AND 6
  AND email IS NOT NULL
  AND (city = 'Delhi' OR name LIKE 'A%');
```

Use a city from your data. Without the parentheses, AND would bind tighter than OR and change the meaning.

3. Exercises (25-40 min)

Seven problems, easy to hard. Write the query, run it, and check that the result makes sense before looking at the answer key.

Exercise 1 (easy). Show the `name` and `email` of customers with ids 2, 4, 6 and 8.

Exercise 2 (easy). List the `name` and `price` of products with ids from 103 to 106.

Exercise 3 (easy). Find all customers whose name contains the letter a. (PostgreSQL: think about uppercase A too.)

Exercise 4 (medium). Find the customers who have no email on file. Show `customer_id` and `name`.

Exercise 5 (medium). Show `customer_id`, `name`, `email` for customers whose id is **not** 1, 2 or 3 and who **do** have an email.

Exercise 6 (medium-hard). From `order_items`, return rows where `quantity` is between 2 and 5 (inclusive) and `product_id` is 101, 102 or 103.

Exercise 7 (hard). Return every customer whose email is **not** a gmail address (look at the emails first to pick the domain), **and** make sure customers with no email at all are included in the result.

4. Stretch Goal

(a) Show every customer's name and email, but display the text `(no email)` instead of NULL. Hint: `COALESCE(email, '(no email)')` returns its first non-NULL argument. We study it properly on Day 22.

(b) Customers 7 and 8 have never placed an order. Without using a JOIN (that is Day 13), write a query that finds them. Hint: a subquery inside IN is a preview of Day 16: `WHERE customer_id NOT IN (SELECT customer_id FROM orders)`. Run it, then explain in one sentence what would go wrong if one order had a NULL `customer_id`.

5. Recap (5 min)

> **One-line takeaway:** `IN` matches a list, `BETWEEN` matches an inclusive range, `LIKE` matches a pattern, and NULL is only ever tested with `IS NULL` / `IS NOT NULL`.

Common mistakes

- Writing `= NULL` instead of `IS NULL` (no error, just zero rows).
- Forgetting that BETWEEN includes both ends, or writing the range backwards.
- Using BETWEEN on timestamp columns and losing the last day.
- Expecting `NOT LIKE` or <> to return NULL rows. They do not; add `OR column IS NULL`.
- Forgetting quotes around text in an IN list, or forgetting that PostgreSQL's LIKE is case-sensitive.
- Mixing AND and OR without parentheses.

Answer Key

Compare your *result*, not just your wording. Different queries can be correct.

Exercise 1

```
SELECT name, email
FROM customers
WHERE customer_id IN (2, 4, 6, 8);
```

4 rows. Equivalent to four ORs. Note that IN works on the id values themselves, so the order of the list does not matter.

Exercise 2

```
SELECT name, price
FROM products
WHERE product_id BETWEEN 103 AND 106;
```

4 rows (103, 104, 105, 106). Both ends are included. `product_id IN (103, 104, 105, 106)` gives the same result.

Exercise 3

```
-- PostgreSQL: catches both 'a' and 'A'
SELECT customer_id, name
FROM customers
WHERE name ILIKE '%a%';

-- Works on both PostgreSQL and SQLite
SELECT customer_id, name
FROM customers
WHERE name LIKE '%a%' OR name LIKE '%A%';
```

The % on both sides means "anything before, anything after". Plain LIKE in PostgreSQL would miss names where the only a is a capital A. SQLite's LIKE is already case-insensitive, so both versions work there.

Exercise 4

```
SELECT customer_id, name
FROM customers
WHERE email IS NULL;
```

Exactly 1 row: customer 7. `email = ''` would be wrong: an empty string is a value, NULL is not.

Exercise 5

```
SELECT customer_id, name, email
FROM customers
WHERE customer_id NOT IN (1, 2, 3)
  AND email IS NOT NULL;
```

4 rows: customers 4, 5, 6 and 8. Customers 4 to 8 pass the NOT IN test, and customer 7 is then removed because their email is NULL.

Exercise 6

```
SELECT *
FROM order_items
WHERE quantity BETWEEN 2 AND 5
  AND product_id IN (101, 102, 103);
```

Each condition must be true (AND). A row with quantity exactly 2 or exactly 5 is included. Check a few rows by eye: every row should have a quantity from 2 to 5 and a product_id of 101, 102 or 103.

Exercise 7

```
SELECT customer_id, name, email
FROM customers
WHERE email NOT LIKE '%@gmail.com'
   OR email IS NULL;
```

Customer 7 must appear in your result. If you wrote only `NOT LIKE`, customer 7 vanishes: NULL NOT LIKE anything is unknown, not true. The extra `OR email IS NULL` brings them back. This is the most common real-world NULL bug.

Stretch goal

```
-- (a)
SELECT name, COALESCE(email, '(no email)') AS email
FROM customers;

-- (b)
SELECT customer_id, name
FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);
```

(a) Customer 7 now shows `(no email)`; the column in the table is unchanged, only the output is. (b) Returns customers 7 and 8. If any `orders.customer_id` were NULL, the list would contain a NULL, every `NOT IN` comparison would become unknown, and the query would return **zero rows**. The safe habit is to filter the subquery with `WHERE customer_id IS NOT NULL`, or to use a LEFT JOIN, which we cover on Day 14.